Write a narrative that explain all Graph steps from first to last step, fluent, simple and logically connected without the bullet points:

In Graph Retrieval-Augmented Generation, the process of building a graph retriever begins long before a user asks a question. The first step is understanding the problem domain and deciding what kind of reasoning the system should eventually perform. For example, in a flight-booking platform, the system may need to reason about airports, airlines, routes, prices, stopovers, and customer preferences. In a predictive maintenance system, the graph may need to represent components, sensors, faults, maintenance actions, and dependencies between subsystems. This initial understanding is extremely important because it determines what entities and relationships will later exist inside the graph.

Once the objective is clear, the system starts collecting raw data from different sources. These sources may include PDFs, manuals, APIs, databases, telemetry streams, maintenance records, schedules, logs, or web documents. At this stage, the information is usually unstructured and inconsistent. Different names may refer to the same entity, measurements may use different units, and some information may be duplicated or incomplete. Therefore, the next stage focuses on cleaning and normalization. The system standardizes terminology, removes duplicates, aligns formats, and resolves inconsistencies so that later stages can build a coherent graph rather than a fragmented one.

After the data is cleaned, large documents are divided into smaller chunks. Although Graph RAG focuses on relationships rather than isolated text retrieval, chunking is still useful because chunks can later become evidence nodes or semantic references linked to entities. These chunks make it easier to trace where information originated and provide grounding evidence during retrieval.

The next major step is entity extraction. Here, the system attempts to identify the important objects, concepts, or actors appearing in the data. In a healthcare application, entities may include symptoms, diseases, medications, and laboratory tests. In an industrial system, entities may include sensors, MOSFETs, cooling pumps, controllers, or fault codes. This extraction can be performed using NLP models, rule-based systems, or modern LLM-based extraction pipelines. At this stage, the graph begins to acquire semantic meaning because the system is no longer looking at raw text but at identifiable real-world concepts.

However, isolated entities alone are not sufficient. The real power of Graph RAG comes from understanding how those entities are connected. Therefore, the next stage is relationship extraction. The system analyzes the text to discover interactions, dependencies, or causal links between entities. For example, a sentence stating that “pump failure caused overheating in inverter module Q3” may generate a relationship connecting the cooling pump to the inverter fault. Similarly, in a travel system, a flight may connect two airports while an airline operates that route. These relationships form the edges of the graph and ultimately create the network through which traversal later occurs.

Once entities and relationships are extracted, the system defines an ontology or schema. This acts as the structural blueprint of the graph. The ontology specifies what types of nodes are allowed, what kinds of relationships may exist between them, and how the domain is logically organized. This is one of the most important design stages because a poorly designed ontology can make traversal inefficient or meaningless. A well-designed ontology ensures that the graph reflects real-world structure rather than random connections.

With the ontology prepared, the actual graph construction process begins. Entities become nodes, relationships become edges, and metadata is attached to both. Gradually, the graph evolves into a connected knowledge structure representing the domain. At this point, the graph is typically stored inside a graph database such as Neo4j, Amazon Neptune, or TigerGraph. These databases are specifically optimized for relationship-based querying and traversal.

Modern Graph RAG systems usually combine graph reasoning with vector semantics, so the next stage involves generating embeddings. Embeddings may be created for nodes, document chunks, summaries, or other graph elements. These embeddings allow the system to perform semantic similarity search in addition to structural traversal. As a result, the retriever becomes hybrid in nature, combining vector search, graph traversal, keyword matching, and metadata filtering into a unified retrieval framework.

Once embeddings are generated, indexing is performed. Vector indexes support semantic search, graph indexes support traversal operations, and metadata indexes enable filtering based on attributes such as time, confidence, severity, or region. After indexing is complete, the offline graph-building phase is largely finished, and the system becomes ready for live query processing.

When a user finally submits a query, the online retrieval phase begins. The system first attempts to understand the query itself. It extracts intent, entities, goals, constraints, and contextual meaning. For example, if the user asks why

an inverter temperature is increasing, the system identifies concepts such as inverter, temperature, and fault diagnosis. These extracted concepts help determine where traversal should begin inside the graph.

The retriever then searches for suitable starting nodes. These entry points may represent components, symptoms, locations, users, routes, or any other entities relevant to the query. Once the starting nodes are identified, the most important phase begins: graph traversal. During traversal, the retriever moves through connected nodes and relationships to discover contextual reasoning paths. Instead of retrieving isolated chunks purely by similarity, the system dynamically explores relationships such as causality, dependency, hierarchy, connectivity, or operational flow.

For example, in an industrial maintenance system, traversal may begin at an overheating inverter, move to the cooling loop, discover reduced coolant flow, identify pump degradation, and finally associate the issue with abnormal vibration readings from a sensor. This creates a multi-hop reasoning chain that explains not only what happened but why it happened. Similarly, in a travel application, traversal may explore airlines, airports, layovers, alliance agreements, prices, and route restrictions to construct an optimized travel recommendation.

As traversal continues, the system extracts a relevant subgraph containing the most useful nodes, relationships, and supporting evidence. This subgraph effectively becomes the contextual memory used for reasoning. Since traversal may discover many possible paths, the retriever then ranks and filters the results to remove irrelevant or low-confidence information. Relevance scoring, reranking models, semantic weighting, and confidence metrics help narrow the retrieved context to the most meaningful subset.

The selected subgraph is then transformed into a structured prompt for the LLM. Instead of sending raw disconnected documents, the system sends organized contextual reasoning paths, connected entities, summaries, and supporting evidence. The LLM therefore receives not just information, but structured relational understanding. This dramatically improves explainability, coherence, and multi-hop reasoning capability.

Finally, the LLM generates the response using the retrieved graph context. The answer may include explanations, recommendations, root-cause analysis, route suggestions, maintenance actions, or diagnostic conclusions. Because the reasoning originated from connected graph structures rather than isolated text fragments, the output is usually more context-aware, explainable, and logically grounded.

In advanced systems, the process does not end there. User feedback, corrections, new observations, or runtime events may later update the graph itself. New entities and relationships can be added continuously, allowing the graph retriever to evolve over time and become progressively more intelligent and contextually aware.

For graph, do we need all vector index, graph index, and metadata index supports?

once graph is embedded, is there any possibility to provide more relationships and entities by uploading some further information. I mean, can additional knowledge be supplied like RAG system?

Yes — in most modern Graph Retrieval-Augmented Generation architectures, all three types of indexing are commonly used together because they solve different retrieval problems. A graph index helps the system efficiently traverse relationships between nodes and edges. A vector index supports semantic similarity search using embeddings. A metadata index enables filtering and structured querying based on attributes such as time, severity, region, device type, confidence score, or document source.

The important point is that these indexes are complementary rather than competing. The graph index answers questions like “what is connected to this entity?” or “what reasoning path exists between these nodes?” The vector index answers questions like “what information is semantically similar to this query?” Meanwhile, the metadata index answers questions such as “show only events from last week” or “retrieve only inverter faults from EV platform A.” Modern enterprise Graph RAG systems usually combine all three because relying on only one retrieval strategy is often insufficient for complex reasoning systems.

A typical retrieval flow therefore becomes hybrid. The query may first undergo semantic vector retrieval to identify candidate entities or chunks. The graph traversal engine then explores relationships around those entities. Metadata filtering may finally narrow the retrieved context based on operational constraints. This layered retrieval strategy is one of the reasons Graph RAG systems are significantly more powerful than classical vector-only RAG systems.

Your second question is extremely important, and the answer is yes: a graph can absolutely continue growing after the initial embedding and indexing stage. In fact, most practical Graph RAG systems are designed to evolve

continuously over time. The graph is not frozen after creation. Additional documents, APIs, logs, manuals, events, or datasets can later be uploaded, processed, and integrated into the existing graph structure.

The process is very similar to how classical RAG systems continuously ingest new documents, but Graph RAG adds an additional structural intelligence layer. When new information is uploaded, the system typically performs another ingestion cycle. It cleans and chunks the new data, extracts new entities, identifies new relationships, and then compares them with the existing graph. If the uploaded information introduces completely new entities, the system creates additional nodes. If it introduces new relationships between existing entities, the graph adds new edges. If it enriches existing entities with more context or evidence, the graph updates properties or attaches additional chunk nodes.

For example, suppose your predictive maintenance graph already contains nodes for an inverter, cooling system, and MOSFETs. Later, you upload a new maintenance report describing a newly observed thermal failure mode. The ingestion pipeline may detect:

- a new fault entity,
- a new causal relationship,
- additional evidence chunks,
- and possibly new sensor patterns.

The graph can then expand dynamically:

```
[Pump                                    Degradation]
    |                                        causes
    ▼
[Coolant                                 Reduction]
    |                                      leads_to
    ▼
[Thermal                                 Oscillation]
```

If “Thermal Oscillation” did not previously exist, a new node is created. If the relationship already existed but new evidence strengthens it, additional chunk nodes or confidence properties may be attached.

The embeddings can also be updated incrementally. Newly added nodes and chunks receive embeddings and are inserted into the vector index without rebuilding the entire graph from scratch. Similarly, graph indexes and metadata indexes are updated incrementally. This incremental ingestion capability is one of the biggest advantages of Graph RAG because it allows the system to continuously learn and evolve as new operational knowledge arrives.

In advanced architectures, the graph may even support streaming updates in near real time. For example:

- new sensor events,
- live airline schedules,
- fresh cybersecurity alerts,
- evolving medical guidelines,
- or newly discovered component dependencies

can continuously modify the graph. This transforms the graph from a static database into a living contextual memory system.

This is also why Graph RAG is especially powerful for domains like:

- predictive maintenance,
- healthcare,
- cybersecurity,
- supply chains,
- intelligent transportation,
- and digital twins,

because the knowledge structure can evolve as the real system evolves.

A very important conceptual distinction is that embeddings alone do not define the graph. The embeddings are only one representation layer used for semantic similarity search. The real graph intelligence comes from the

relationships, topology, traversal paths, and connected reasoning structures. Therefore, even after embeddings are generated, the graph itself remains fully extensible and continuously updateable.

what is provenance here:
In this context, **provenance** means:
information about where a graph relationship came from, how it was created, and how trustworthy it is.
In other words:
provenance is the traceable origin and history of a node or edge.

Note:
In distributed systems, "**synchronous**" does **not** mainly mean:

- "operations happen at the same time"
- or "multiple systems process simultaneously"

Instead, synchronous means:
one operation directly waits for another operation to finish before continuing.
So the key idea is:
request → wait → response → continue
The caller is blocked until the callee finishes.

In Your Architecture Narrative

A more technically precise wording could be:
"Synchronous workflows are operations where the request lifecycle, execution ordering, and transactional consistency require the system to wait for deterministic completion before proceeding to the next stage."

what is a reconciliation workers? what is the difference between reconciliation workers and Delayed retry? do these all do the same things (Apache Kafka, Azure Event Hubs, Amazon Kinesis, or cloud service buses)?
A **reconciliation worker** is a background service that checks whether different systems ended up in the same correct state.
For example, after a flight booking:
Payment system says: paid
Airline booking API says: pending
Your database says: awaiting confirmation

A reconciliation worker periodically checks these records, compares them, and fixes or escalates the mismatch. Its job is **state correction and consistency checking**.
A **delayed retry** is different. It simply tries a failed operation again later.
Booking API timed out
→ wait 30 seconds
→ try again
→ if still failing, try later or move to dead-letter queue

So:
Delayed retry = repeat a failed action later
Reconciliation worker = inspect final states and repair inconsistencies

They often work together. A retry may eventually succeed, but reconciliation checks whether the whole workflow actually became consistent.
Kafka, Azure Event Hubs, Amazon Kinesis, and cloud service buses are related, but they do **not** all do exactly the same thing.

Kafka, Event Hubs, and Kinesis are mainly **event streaming platforms**. They are good when many events continuously flow through the system, such as booking-created, payment-confirmed, flight-delayed, price-changed, or graph-updated events. They allow multiple consumers to read the same event stream independently.
Cloud service buses and queues such as Azure Service Bus, AWS SQS, or RabbitMQ are mainly **message/workflow queues**. They are good for reliable task execution, retries, background jobs, and worker processing, such as send notification, retry booking confirmation, update index, or run reconciliation.
A simple distinction is:
Event streaming = publish facts/events for many services to react to
Queue/service bus = assign work/tasks for workers to process

For your flight-booking system, you might use a **service bus or queue first** for notifications, retries, ingestion, and reconciliation. Kafka/Event Hubs/Kinesis become more useful later if the platform has high-volume continuous events, analytics, monitoring streams, or many independent consumers.
Also, I noted what you defined as SYNC actions (an action wait for another before doing job) is similar as stateful. So, how do you differentiate the sync/async from stateful/stateless?
Excellent observation.
They are related concepts, and in many systems they overlap, which is why people often confuse them. But they describe **different dimensions** of system behaviour.
The cleanest way to separate them is this:

Concept	Main Question
Sync vs Async	"Do we WAIT?"
Stateful vs Stateless	"Do we REMEMBER?"

That is the fundamental distinction.

1. Sync vs Async → Execution Behaviour

This describes:
how operations execute and coordinate in time.

Synchronous

request
↓
wait
↓
response
The workflow pauses until the operation finishes.

Asynchronous

request
↓
continue immediately
↓
result arrives later
The workflow does not wait actively.

2. Stateful vs Stateless → Memory Behaviour

This describes:
whether the system stores workflow/session memory between requests.

Stateful

The system remembers previous context.
Example:
user selected:
- London → Tokyo
- ANA preferred
- business class
- payment pending
The workflow state is preserved.
Usually stored in:

- Redis
- PostgreSQL
- session store
- workflow engine

Stateless

The service remembers nothing between requests.
Every request is independent.
Example:
GET /search?q=Tokyo
The API processes the request and forgets it afterward.

Final Senior-Level Definition

You could explain it in interviews like this:
“Synchronous versus asynchronous describes execution coordination and waiting behaviour, while stateful versus stateless describes persistence of workflow or session memory. Although many transactional workflows are both synchronous and stateful, the two concepts represent different architectural dimensions.”

End-to-End Steps for Building a Graph Retriever in Graph Retrieval-Augmented Generation

A Graph Retriever pipeline usually has two major phases:

1. **Offline Graph Construction Phase**
2. (building the knowledge graph)
3. **Online Retrieval & Traversal Phase**

(query-time reasoning and retrieval)

BIG PICTURE FLOW

Raw Data
↓
Ingestion
↓
Chunking & Cleaning
↓
Entity Extraction
↓
Relationship Extraction
↓
Ontology / Schema Design
↓
Graph Construction
↓
Graph Storage & Indexing
↓
Embedding Generation
↓
Hybrid Retrieval Layer
↓
Graph Traversal Engine
↓
Subgraph Assembly
↓
Prompt Construction
↓
LLM Reasoning
↓
Final Response

PHASE 1 — DEFINE THE PROBLEM

Before building the graph:

Define:

Item	**Example**
Domain	Flight booking
Objective	Route reasoning
Node types	Airport, airline, flight
Edge types	connects_to, operated_by
Queries	“cheapest route avoiding overnight stops”

PHASE 2 — DATA INGESTION

Collect raw data.
Sources may include:

Source	Example
PDFs	manuals
APIs	flight data
Databases	maintenance records
Logs	sensor logs
Tables	schedules
Web data	policies

PHASE 3 — CLEANING & NORMALIZATION

Prepare the data.
Tasks:

- remove duplicates
- standardize names
- normalize units
- clean formatting
- resolve inconsistencies

Example:
"LHR" "London" "Heathrow" → normalized to one entity "Airport" "Heathrow"

PHASE 4 — CHUNKING

Split large documents into smaller units.
Example:
Maintenance Manual → Section → Paragraph → Chunk

Graph RAG still often uses chunking because:

- chunks become evidence nodes
- chunks may later receive embeddings

PHASE 5 — ENTITY EXTRACTION

Identify important entities.
Usually done using:

- NLP
- NER models
- LLM extraction
- rules/patterns

Example

Text:
MOSFET Q3 overheated due to coolant failure.

Extract:

Entity	Type
MOSFET Q3	Component
coolant failure	Fault

PHASE 6 — RELATIONSHIP EXTRACTION

Identify relationships between entities.

Example

MOSFET Q3 —affected_by—> Coolant Failure

Methods:

Method	Description
Rules	Regex/pattern
Dependency parsing	NLP
LLM extraction	Most modern
Knowledge templates	Domain ontology

PHASE 7 — ONTOLOGY / SCHEMA DESIGN

This is VERY important.
Ontology defines:

- node types
- edge types
- allowed relationships
- hierarchy

Example Ontology

Node Types:
- Component
- Sensor
- Fault
- Maintenance Action

Edge Types:
- connected_to
- monitored_by
- causes
- mitigated_by

PHASE 8 — GRAPH CONSTRUCTION

Now create the graph.

Nodes

Node:
{
id: "MOSFET_Q3",
type: "Component"
}

Edges

Edge:
{
source: "Coolant_Failure",
relation: "causes",
target: "MOSFET_Q3"
}

PHASE 9 — GRAPH STORAGE

Store graph in graph database.
Common options:

Database	Notes
Neo4j	most popular
TigerGraph	high-scale
Amazon Neptune	AWS-native
ArangoDB	graph + docs

PHASE 10 — CREATE EMBEDDINGS

Modern Graph RAG usually combines:

- graph structure
- vector embeddings

Generate embeddings for:

Item	Why
nodes	semantic search
chunks	retrieval
summaries	reasoning
edges sometimes	semantic relations

PHASE 11 — INDEXING

Build retrieval indexes.
Possible indexes:

Type	Purpose
Vector index	semantic retrieval
Graph index	traversal
Keyword index	BM25
Metadata index	filtering

This becomes:

Hybrid Retrieval

Vector + Graph + Keyword + Metadata

PHASE 12 — QUERY UNDERSTANDING

At runtime:
User asks question.
Example:
"Why is inverter temperature rising?"

System extracts:

Extracted	Value
entity	inverter
issue	temperature
intent	diagnosis

PHASE 13 — START NODE IDENTIFICATION

Find relevant graph entry points.
Example:
Start Nodes:
- Inverter
- Temperature Sensor
- Cooling System

PHASE 14 — GRAPH TRAVERSAL

This is the graph retriever core.
Traverse through connected entities.
Example:
Inverter
→ Cooling Loop
→ Pump Failure
→ Reduced Coolant Flow
→ MOSFET Overheating

Traversal strategies:

Strategy	Use
BFS	nearby context
DFS	deep reasoning
weighted traversal	priorities
semantic traversal	AI-guided

PHASE 15 — SUBGRAPH EXTRACTION

Collect relevant connected region.
Example:
Relevant Subgraph:
- inverter
- coolant loop
- pump

- MOSFET
- sensor alerts

This subgraph becomes contextual evidence.

PHASE 16 — CONTEXT RANKING & FILTERING

Not all traversed nodes are useful.
Apply:

- reranking
- scoring
- relevance filtering
- confidence weighting

PHASE 17 — PROMPT ASSEMBLY

Build structured prompt.
Example:

```
Relevant Components:
- MOSFET Q3 overheating
- Coolant flow reduction
- Pump efficiency degraded

Reasoning Path:
Pump Failure → Reduced Cooling → Thermal Stress
```

PHASE 18 — LLM REASONING

Now send:

- query
- retrieved graph context
- reasoning paths
- supporting chunks

to the LLM.

PHASE 19 — RESPONSE GENERATION

LLM produces:

- answer
- explanation
- recommendations
- reasoning trace

PHASE 20 — OPTIONAL FEEDBACK LOOP

Capture:

- user feedback
- successful traversals
- corrections
- new relations

Then update graph.

FINAL MENTAL MODEL

OFFLINE BUILD PHASE

Data
→ Entity Extraction
→ Relationship Extraction
→ Ontology
→ Graph Build
→ Embeddings
→ Indexing

ONLINE QUERY PHASE

User Query
→ Query Understanding
→ Start Node Detection
→ Graph Traversal
→ Subgraph Extraction
→ Prompt Building
→ LLM Reasoning
→ Final Answer

KEY INSIGHT (MOST IMPORTANT)

Classical RAG retrieves:
similar chunks

Graph RAG retrieves:
connected reasoning structures

That is the fundamental difference.